Table of Contents

Governing Equations
Momentum Equation
Continuity Equation
Turbulence Equation
Reconstruction Scheme
Face Reconstruction
Linear Interpolation
Non-Orthogonal Correction
Skewness Correction
Gradient Reconstruction
Green-Gauss Gradient
Least Squares Gradient
Reconstruction Algorithm
Boundary Conditions
Transport Equation
Diffusion Term
Minimum Correction
Orthogonal Correction
Over-Relaxed
Matrix Assembly
Convection Term
Barth–Jespersen Limiter
Venkatakrishnan Limiter
Matrix Assembly
Temporal Term
Matrix Assembly
Source Term
Matrix Assembly
Pressure Equation
Matrix Assembly
Turbulence Model
Spalart-Allmaras Model
Wall Function
Spalding's Law
Newton-Raphson Iteration
Source Assembly
Solver Algorithm
Appendix

Governing Equations

Momentum Equation

is supplied by the Spalart-Allmaras model; recovers the laminar solver.

Continuity Equation

Turbulence Equation

, , and are defined in Spalart-Allmaras model.

Reconstruction Scheme

Face Reconstruction

Linear Interpolation

Non-Orthogonal Correction

Skewness Correction

Gradient Reconstruction

Green-Gauss Gradient

Least Squares Gradient

Reconstruction Algorithm

1. Interpolate face scalar.
2. Apply boundary conditions.
3. Compute cell gradient .
4. Interpolate face gradient .
5. Correct non-orthogonality for face gradient .
6. Correct skewness for face scalar.
7. Apply boundary conditions.

Boundary Conditions

Field Body Face Farfield Face
(Inflow, )

Farfield Face
(Outflow, )

extrapolated from cell

extrapolated from cell

extrapolated from cell extrapolated from cell

from pressure correction

extrapolated from cell

(integrate to wall)
(wall function)

extrapolated from cell extrapolated from cell

Transport Equation

The momentum and turbulence equations share the form

with the following specialisations:

Equation

Momentum

Turbulence

Diffusion Term

Minimum Correction

Orthogonal Correction

Over-Relaxed

Matrix Assembly

Element Contribution Contribution

Interior Face

Body Face

Farfield Face (Inflow)

Farfield Face (Outflow) — —

Convection Term

Barth–Jespersen Limiter

Venkatakrishnan Limiter

Matrix Assembly

Element Contribution Contribution

Interior Face

Body Face — —

Farfield Face (Inflow) —

Farfield Face (Outflow)

Temporal Term

Matrix Assembly

Element Contribution Contribution

Cell

Source Term

Matrix Assembly

Element Contribution Contribution

Cell —

Pressure Equation

* see Appendix for more information

Matrix Assembly

Element Contribution Contribution

Interior Face

Body Face — —

Farfield Face (Inflow) —

Farfield Face (Outflow)

Turbulence Model

Spalart-Allmaras Model

Constant Value

Wall Function

Spalding's Law

(implicit in)

Newton-Raphson Iteration

(solve for)

Source Assembly

Temporal, convection, and diffusion follow the Transport Equation assembly with and . Production and cross-diffusion
are added to, while destruction is linearised semi-implicitly to put a positive coefficient on the diagonal.

Element Contribution Contribution

Cell (Production) —

Cell (Destruction) —

Cell (Cross-Diffusion) —

Solver Algorithm

Initialise,,, and (if turbulence is enabled), from the initial and boundary conditions.

For each time step:

1. Save ; reclassify each farfield face as inflow ( ) or outflow ( ).

2. Momentum predictor:

1. For : reconstruct from; save .
2. Assemble (shared by,) from temporal, diffusion ( ), and convection.
3. For :
1. Assemble from temporal, diffusion, and convection; save .
2. Add source to; solve .

3. Compute ; reconstruct from.

4. Assemble pressure matrix from.

5. Pressure corrector (repeat times):

1. Compute ; reconstruct from.
2. Compute from.
3. Assemble; solve .
4. Correct ; reconstruct from.
5. Correct .

6. Turbulence corrector (if enabled):

1. Reconstruct from; evaluate closure, from ,,.
2. Assemble from temporal, diffusion ( ), convection, and destruction.
3. Assemble from temporal, diffusion, convection, production, and cross-diffusion.
4. Solve ; clamp .
5. Update ; refresh with body-face override ( or ).

Appendix

Pressure Equation

Let denote the cell-centred velocity. The discrete momentum equation at cell separates the pressure source from the rest
of the right-hand side:

Solving for and grouping the pressure-free part as with prefactor :

Note that the off-diagonal sum in can be rewritten using the matrix-vector product. Since

we have

Substituting into the definition of:

The code uses this latter form because is a single matrix-vector multiplication.

Vectorising over all cells and imposing continuity yields the pressure Poisson equation:

Solving this Poisson equation gives, which projects onto the divergence-free constraint via .

Rhie-Chow Residual

On a staggered grid the face mass flux is updated directly via the face-centred relation:

On a collocated grid the cell velocity is updated and then interpolated to the face:

The face flux from this interpolation differs from the face-centred update:

Rhie-Chow removes this residual by computing via a face-centred update directly, recovering staggered-grid behaviour on a collocated
mesh.

Rhie-Chow Correction

For unsteady flow, the predictor RHS without the pressure source contains a temporal contribution:

Dividing by produces a corresponding term in:

Interpolating to the face and projecting onto:

Replacing the collocated face-velocity contribution with the face-centred mass flux from the previous step gives the Rhie-Chow-
corrected predictor flux:

The correction replaces the collocated-interpolated face velocity contribution with the face-centred (Rhie-Chow-corrected)
mass flux from the previous step.

∂ϕ
∂t
temporal

+ ∇ ⋅ (ϕu)

convection

= ∇ ⋅ [(ν + νt)∇ϕ]

diffusion

− ∇p ⋅ e^ϕ

source



        

ϕ ∈ {u, v} ν =

1
Re

νeff = ν + νt e^ϕ ∈ {

^i,^j}

νt νt = 0

∇ ⋅ u = 0

∂

~ν
∂t
temporal

+ ∇ ⋅ (~νu)

convection

= ∇ ⋅ (

ν + ~ν

σ

∇~ν)

diffusion

+ cb1

~S~ν
production

− cw1fw(

~ν
d

)

2

destruction

+

cb2
σ

∥∇~ν∥

2

cross-diffusion



     

     

  

~ν ← max(~ν, 0)

~S fw d

–

ϕf = {

wfϕC + (1 − wf)ϕN interior face
ϕC boundary face

–

(∇ϕ)f = {

wf (∇ϕ)C + (1 − wf)(∇ϕ)N interior face
(∇ϕ)C boundary face

wf =

∥dNf∥
∥dCf∥ + ∥dNf∥

(∇ϕ)f = (∇ϕ)f + [

ϕN − ϕC
∥dCN ∥

− (∇ϕ)f ⋅ d^ CN]d^

–

CN

ϕf = ϕf + (∇ϕ)f ⋅ df

′

–

f

(∇ϕ)C =

1

VC ∑

f

ϕfSf

J = ∑

N

w

2

CN [(∇ϕ)C ⋅ dCN − (ϕN − ϕC)]

2

wCN =

1

∥dCN ∥

MC(∇ϕ)C = rC

MC = ∑

N

w

2

CN dCN d

⊤

CN rC = ∑

N

w

2

CN (ϕN − ϕC)dCN

(∇ϕ)C = M−1

C

rC

–

ϕf

–

(∇ϕ)C

–

(∇ϕ)f
(∇ϕ)f
ϕf

m˙ f ≤ 0 m˙ f > 0

u 0 u∞

h 0 u∞

p 0

m˙ f 0 u∞ ⋅ Sf

~ν 0

~ν∞

ν + νt ν
νwall

∂ϕ
∂t
temporal

+ ∇ ⋅ (ϕu)

convection

− ∇ ⋅ (Γ∇ϕ)

diffusion

= Sϕ

source



      

ϕ Γ Sϕ

u, v ν + νt −∇p ⋅ e^ϕ

~ν (ν + ~ν)/σ cb1

~S~ν − cw1fw(~ν/d)

2 + (cb2/σ)∥∇~ν∥

2

∫

V

∇ ⋅ (Γ∇ϕ)dV = ∑
f

Γf (∇ϕ)f

⋅ Sf

Sf = Ef + Tf

Γf (∇ϕ)f

⋅ Sf = Γf∥Ef∥

ϕN − ϕC
∥dCN ∥

implicit

+ Γf (∇ϕ)f

⋅ Tf

explicit

  

  

Ef = (Sf

⋅ d^ CN)d^ CN

Ef = ∥Sf∥d^ CN

Ef =

∥Sf∥

2

Sf

⋅ d^ CN

d^ CN

A b

A[C, C], A[N, N] += +Γf∥Ef∥/∥dCN ∥
A[C, N], A[N, C] += −Γf∥Ef∥/∥dCN ∥

b[C] += +Γf (∇ϕ)f

⋅ Tf
b[N] += −Γf (∇ϕ)f ⋅ Tf

A[C, C] += +Γf∥Ef∥/∥dCf∥ b[C] += +Γf∥Ef∥/∥dCf∥ ⋅ ϕf + Γf (∇ϕ)f

⋅ Tf

A[C, C] += +Γf∥Ef∥/∥dCf∥ b[C] += +Γf∥Ef∥/∥dCf∥ ⋅ ϕf + Γf (∇ϕ)f

⋅ Tf

∫

V

∇ ⋅ (ϕu)dV = ∑
f

m˙ fϕf

m˙ f = uf ⋅ Sf

U = {

C m˙ f ≥ 0
N m˙ f < 0

m˙ fϕf = m˙ fϕU

implicit

+ m˙ fΨU(∇ϕ)U ⋅ dUf

explicit

     

ΨU ∈ [0, 1] Δf = (∇ϕ)U ⋅ dUf

ϕmax = max(ϕU, max

N

ϕN) ϕmin = min(ϕU, min

N

ϕN)

Δmax = ϕmax − ϕU Δmin = ϕmin − ϕU

ΨU = min
f

⎧

⎪

⎨

⎩

min(1,

Δmax
Δf

) Δf > 0

min(1,

Δmin
Δf

) Δf < 0
1 Δf = 0

Δ∗ = {

Δmax Δf > 0
Δmin Δf < 0

K ∈ [1, 5]

ϵ
2 = (KV

1/ndim
U

)

3

ΨU = min
f

Δ2

∗ + 2ΔfΔ∗ + ϵ

2

Δ2

∗ + 2Δ2

f + ΔfΔ∗ + ϵ

2

A b

A[C, C] += + max(m˙ f, 0)
A[C, N] += + min(m˙ f, 0)
A[N, N] += − min(m˙ f

, 0)
A[N, C] += − max(m˙ f, 0)

b[C] += −m˙ fΨU(∇ϕ)U ⋅ dUf
b[N] += +m˙ fΨU(∇ϕ)U ⋅ dUf

b[C] += − min(m˙ f, 0) ⋅ ϕf

A[C, C] += + max(m˙ f, 0) b[C] += − max(m˙ f, 0)ΨC(∇ϕ)C ⋅ dCf

∫

V

∂ϕ
∂t

dV ≈ VC

ϕ

(n+1)
C − ϕ

(n)
C

Δt

A b

A[C, C] += VC/Δt b[C] += (VC/Δt)ϕ

(n)
C

∫

V

SϕdV ≈ VCSϕ

A b

b[C] += VCSϕ

∇ ⋅ (Γ∇p) = ∇ ⋅ h

ΓC =

VC

aC

hC = uC +

(b−p)C − (Au)C

aC

∫

V

∇ ⋅ (Γ∇p)dV = ∑
f

Γf (∇p)f ⋅ Sf

∫

V

∇ ⋅ hdV = ∑
f

hf ⋅ Sf

m˙

∗

f = {

hf

⋅ Sf +

Γf

Δt (m˙

(n)
f − u

(n)
f

⋅ Sf) interior face
hf ⋅ Sf boundary face

A b

A[C, C], A[N, N] += +Γf∥Ef∥/∥dCN ∥ A[C, N], A[N, C] += −Γf∥Ef∥/∥dCN ∥

b[C] += −m˙

∗

f + Γf (∇p)f ⋅ Tf
b[N] += +m˙

∗

f − Γf (∇p)f ⋅ Tf

b[C] += −u∞ ⋅ Sf

A[C, C] += +Γf∥Ef∥/∥dCf∥ b[C] += −hf

⋅ Sf + Γf (∇p)f

⋅ Tf

χ =

~ν
ν

fv1 =

χ

3

χ3 + c

3
v1

fv2 = 1 −

χ

1 + χfv1

νt = ~νfv1

Ω =

∂v
∂x

−

∂u
∂y

~S = max(Ω +

~νfv2
(κd)

2

∣ ∣, csΩ) d ≡ nearest wall distance

r = min(

~ν

~S(κd)

2
, rmax) g = r + cw2(r

6 − r) fw = g(

1 + c

6
w3

g

6 + c

6
w3

)

1/6

σ 2/3

κ 0.41

cb1 0.1355

cb2 0.622

cv1 7.1

cw1 cb1/κ

2 + (1 + cb2)/σ

cw2 0.3

cw3 2.0

cs 0.3

rmax 10

uτ

y

+ = u

+ +

1
E

[exp(κu

+

) − 1 − κu

+ −

(κu

+)

2

2

−

(κu

+)

3

6

]
y

+ =

uτyc

ν

u

+ =

u∥

uτ

E = 9.8

uτ

u

(0)

τ = √

νu∥
yc

uτ ← max(uτ − f/f

′

, ϵ)

f(uτ) = y

+ − u

+ −

1
E

[exp(κu

+

) − 1 − κu

+ −

(κu

+)

2

2

−

(κu

+)

3

6

]
f

′

(uτ) =

yc

ν

+

u∥

u

2
τ

+

κu
+ [exp(κu

+) − 1 − κu

+ − (κu

+)

2/2]
E uτ

νwall =

u

2

τyc
u∥

ϕ = ~ν Γf = (ν + ~νf)/σ
b A

A b

b[C] += +cb1

~SC

~νCVC

A[C, C] += +cw1fw,C

~νCVC/d

2
C

b[C] += +(cb2/σ)∥(∇~ν)C∥

2VC

u p m˙ f

~ν νt

n

m˙

(n)
f ← m˙ f m˙ f ≤ 0 m˙ f > 0

ϕ ∈ {u, v} ϕf ϕ ϕ

(n)
f ← ϕf
A u v Γf = ν + νt
ϕ ∈ {u, v}
b b−p ← b
− ∇p ⋅ e^ϕ b Aϕ = b

ΓC = VC/ACC Γf Γ

Ap Γf

npiso

hC = uC + ((b−p)C − (Au)C)/ACC hf h
m˙

∗

f hf
b App = b
m˙ f ← m˙

∗

f − Γf (∇p)f

⋅ Sf pf p
uC ← hC − ΓC(∇p)C

~νf

~ν

~S fw

~ν

(n) Ω d
A~ν Γf = (ν + ~νf)/σ
b
A~ν

~ν = b ~ν ← max(~ν, 0)
νt = ~νfv1 (νeff)f = ν + (νt)f ν νwall

uC:= [uC, vC]⊤ C

aCuC + ∑

N∈C

aN uN = bC = (b−p)C − (∇p)CVC

uC hC ΓC:= VC/aC

uC =

(b−p)C
aC

− ∑

N∈C

aN
aC

uN −

VC
aC

(∇p)C

hC:=

(b−p)C

aC

− ∑
N∈C

aN
aC

uN ΓC:=

VC
aC

uC = hC − ΓC(∇p)C

hC Au

(Au)C = aCuC + ∑

N∈C

aN uN

∑
N∈C

aN
aC

uN =

(Au)C

aC

− uC

hC

hC =

(b−p)C
aC

− ∑

N∈C

aN
aC

uN = uC +

(b−p)C − (Au)C
aC

Au

∇ ⋅ u = 0

u = h − Γ∇p

∴ ∇ ⋅ (Γ∇p) = ∇ ⋅ h

p u u = h − Γ∇p

m˙ f ← m˙

∗

f − Γf (∇p)f ⋅ Sf

uC ← hC − ΓC(∇p)C

uf ← wfuC + (1 − wf)uN

m˙ f − uf

⋅ Sf ≠ 0

m˙ f

(b−p)C =

VC
Δt

u

(n)
C + convection + diffusion

aC ΓC/Δt hC

hC =

(b−p)C

aC

− ∑
N∈C

aN
aC

uN =

ΓC
Δt

u

(n)
C + rest

Sf

hf =

Γf
Δt

u

(n)
f + (rest)f

hf ⋅ Sf =

Γf
Δt

(u

(n)
f

⋅ Sf) + (rest)f ⋅ Sf
u

(n)
f

⋅ Sf m˙

(n)
f

∴ m˙

∗

f ← hf

⋅ Sf +

Γf
Δt

(m˙

(n)
f − u

(n)
f

⋅ Sf)
m˙

(n)
f − u

(n)
f

⋅ Sf